

Semana 2 – Guía de prácticas (2h)

Objetivo: Diseñar y validar un formulario accesible con semántica correcta (landmarks), ayudas y errores anunciables, validación nativa y reglas propias (contraseñas, fechas, email “ya existente”), y verificación rápida WCAG.

Requisitos previos: VS Code + Live Server (Semana 1), nociones básicas de HTML/CSS/JS.

Estructura del proyecto

```
semana2-formularios/  
├── public/  
│   └── index.html  
├── styles/  
│   └── styles.css  
└── scripts/  
    └── app.js
```

Abre la carpeta en VS Code. Usa Live Server sobre public/index.html.

Hora 1 (≈60 min) — Maquetado semántico + validación nativa

1) Crea public/index.html

```
<!doctype html>  
<html lang="es">  
<head>  
  <meta charset="utf-8" />  
  <title>Registro de Cuenta – Semana 2 (Hotfix)</title>  
  <meta name="viewport" content="width=device-width, initial-scale=1" />  
  <meta name="description" content="Formulario accesible con validación nativa y personalizada." />  
  <link rel="stylesheet" href="../styles/styles.css" />  
</head>  
<body>  
  <!-- Skip link para teclado/lectores -->  
  <a class="skip-link" href="#contenido">Saltar al contenido</a>  
  
  <header>  
    <h1>Plataforma Académica</h1>  
    <nav aria-label="Principal">  
      <ul>  
        <li><a href="#" aria-current="page">Inicio</a></li>  
        <li><a href="#formulario">Registro</a></li>  
      </ul>  
    </nav>  
  </header>  
  
  <main id="contenido">  
    <!-- Resumen de errores (JS lo rellena si hace falta) -->
```

```

<section aria-labelledby="t-resumen" class="visually-group" id="resumen-seccion">
  <h2 id="t-resumen">Resumen de errores</h2>
  <div id="form-summary" role="alert" aria-live="polite"></div>
</section>

<section id="formulario" aria-labelledby="t-form">
  <h2 id="t-form">Registro de cuenta</h2>

  <!-- Usamos novalidate para controlar mensajes,
    pero dejamos atributos nativos en el resto de campos -->
  <form id="f-registro" novalidate aria-describedby="ayuda-general">
    <p id="ayuda-general">Completa los campos obligatorios. Navega con Tab y Enter.</p>

    <!-- Datos personales -->
    <fieldset>
      <legend>Datos personales</legend>

      <label for="nombre">Nombre</label>
      <input id="nombre" name="nombre" type="text" required minlength="2"
        autocomplete="given-name" aria-describedby="hint-nombre nombre-err">
      <small id="hint-nombre">Tal como figura en tu documento (mínimo 2 caracteres).</small>
      <span id="nombre-err" class="error" role="alert" hidden></span>

      <label for="apellidos">Apellidos</label>
      <input id="apellidos" name="apellidos" type="text" required minlength="2"
        autocomplete="family-name" aria-describedby="hint-apellidos apellidos-err">
      <small id="hint-apellidos">Mínimo 2 caracteres.</small>
      <span id="apellidos-err" class="error" role="alert" hidden></span>

      <label for="email">Correo institucional</label>
      <input id="email" name="email" type="email" required
        autocomplete="email" aria-describedby="hint-email email-status email-err">
      <small id="hint-email">Formato: usuario@dominio.</small>
      <!-- Estado no intrusivo (validación asíncrona simulada) -->
      <small id="email-status" role="status" aria-live="polite"></small>
      <span id="email-err" class="error" role="alert" hidden></span>

      <label for="tel">Teléfono</label>
      <input id="tel" name="tel" type="tel" inputmode="tel"
        autocomplete="tel" aria-describedby="hint-tel">
      <small id="hint-tel">Incluye código de área si aplica.</small>
    </fieldset>

    <!-- Acceso -->
    <fieldset>
      <legend>Acceso</legend>

      <label for="pass">Contraseña</label>
      <!-- SIN minlength: la longitud se valida en JS -->

```

```

        <input id="pass" name="pass" type="password" required
            aria-describedby="hint-pass pass-err pass-strength-text">
        <small id="hint-pass">Mínimo 8 caracteres. Combina mayúsculas, números y
símbolos.</small>
        <progress id="pass-strength" value="0" max="4" aria-hidden="true"></progress>
        <small id="pass-strength-text" role="status" aria-live="polite"></small>
        <span id="pass-err" class="error" role="alert" hidden></span>

        <label for="pass2">Confirmar contraseña</label>
        <input id="pass2" name="pass2" type="password" required
            aria-describedby="pass2-err">
        <span id="pass2-err" class="error" role="alert" hidden></span>
    </fieldset>

    <!-- Preferencias -->
    <fieldset>
        <legend>Preferencias</legend>
        <label><input type="radio" name="contacto" value="correo" checked> Contacto por
correo</label>
        <label><input type="radio" name="contacto" value="telefono"> Contacto por
teléfono</label>

        <label><input type="checkbox" id="news" name="news" value="si"> Deseo recibir
boletines</label>
    </fieldset>

    <!-- Aceptación -->
    <fieldset>
        <legend>Condiciones</legend>
        <label>
            <input type="checkbox" id="acepto" name="acepto" required aria-describedby="acepto-
err">

            Acepto términos y condiciones
        </label>
        <span id="acepto-err" class="error" role="alert" hidden></span>
    </fieldset>

    <button type="submit">Crear cuenta</button>
</form>
</section>
</main>

<footer>
    <p>© 2025. Curso Desarrollo en Plataformas.</p>
</footer>

<!-- cache-busting para evitar JS viejo en caché -->
<script src="../scripts/app.js?v=hotfix5"></script>
</body>

```

```
</html>
```

2) Crea styles/styles.css

```
/* Base y foco visible */
* { box-sizing: border-box; }
body { font-family: system-ui, -apple-system, Segoe UI, Roboto, sans-serif; line-height: 1.5;
margin: 0; }
header, main, footer { padding: 1rem; }
:focus-visible { outline: 3px solid #000; outline-offset: 2px; }

/* Skip link accesible */
.skip-link { position: absolute; left: -999px; top: -999px; }
.skip-link:focus { position: static; background:#eee; padding:.5rem; }

/* Navegación simple */
nav ul { list-style: none; padding: 0; display: flex; gap: 1rem; }

/* Formulario */
form { max-width: 780px; }
fieldset { border: 1px solid #ddd; padding: 1rem; margin-bottom: 1rem; border-radius: .5rem; }
legend { font-weight: 600; }
label { display: block; margin-top: .75rem; }
input, select, textarea { width: 100%; padding: .6rem; margin-top: .25rem; border-radius: .4rem;
border: 1px solid #bbb; }
button { margin-top: 1rem; padding: .7rem 1rem; border-radius: .5rem; border: 1px solid #333;
background: #fff; cursor: pointer; }

/* Estados nativos (siguen aplicando a otros campos) */
input:required:invalid, select:required:invalid { border-color: #c00; }
input:required:valid, select:required:valid { border-color: #0a0; }

/* Mensajes */
.error[hidden] { display: none; }
.error { color: #a40000; margin-top: .25rem; }

/* Resumen de errores */
.visually-group { margin-bottom: 1rem; }
#form-summary p { margin: 0 0 .25rem 0; }
#form-summary ul { margin: .25rem 0 0 1rem; }
#form-summary a { text-decoration: underline; }

/* Accesibilidad visual extra */
@media (prefers-reduced-motion: reduce){
  * { animation-duration: .01ms !important; animation-iteration-count: 1 !important; }
}
```

3) Verifica en Live Server

- Recorre con Tab/Shift+Tab: foco visible, orden lógico.
- Prueba validación nativa: deja vacíos obligatorios, prueba email mal formado.

Hora 2 (≈60 min) — Validación personalizada (JS) + accesibilidad

4) Crea scripts/app.js

```
// ===== Identificador de versión (para confirmar que cargó este archivo) =====
console.log('%cSemana2 app.js HOTFIX v5', 'color:green', new Date().toISOString());

// ===== Utilidades de mensajes =====
function plural(n, uno, otros){ return n === 1 ? uno : otros; }

const messages = {
  valueMissing:    'Completa este campo.',
  typeMismatch:    'Revisa el formato (p. ej., usuario@dominio).',
  patternMismatch: 'Sigue el formato indicado.',
  tooShort:        el => {
    const rest = el.minLength - el.value.length;
    return `Añade ${rest} ${plural(rest, 'carácter', 'caracteres')} más.`;
  },
  tooLong:         el => `Reduce a ${el.maxLength} caracteres.`,
  rangeUnderflow:  el => `El valor mínimo es ${el.min}.`,
  rangeOverflow:   el => `El valor máximo es ${el.max}.`,
  stepMismatch:    'Ajusta al paso permitido.',
  badInput:        'Usa solo números en este campo.',
  customError:     el => el.validationMessage || 'Revisa este dato.'
};

function nativeMessage(el){
  const v = el.validity;
  if (v.valid) return '';
  for (const k of Object.keys(messages)){
    if (v[k]){
      const m = messages[k];
      return typeof m === 'function' ? m(el) : m;
    }
  }
  return 'Revisa este dato.';
}

function showError(el, msg){
  const ids = (el.getAttribute('aria-describedby')||'').split(' ');
  const errId = ids.find(id => id && id.endsWith('-err'));
  const errEl = errId ? document.getElementById(errId) : null;

  el.setCustomValidity(msg||'');
  if (msg){
    el.setAttribute('aria-invalid', 'true');
    if (errEl){ errEl.textContent = msg; errEl.hidden = false; }
  } else {
    el.removeAttribute('aria-invalid');
  }
}
```

```

    if (errEl){ errEl.hidden = true; errEl.textContent = ''; }
  }
}

function renderSummary(form, invalidFields){
  const box = document.getElementById('form-summary');
  if (!invalidFields.length){ box.innerHTML = ''; return; }
  const items = invalidFields.map(({el, msg})=>{
    const label = form.querySelector(`label[for="${el.id}"]`);
    const texto = label ? label.textContent.trim() : el.name || el.id;
    return `<li><a href="#${el.id}">${texto}</a> ${msg}</li>`;
  }).join('');
  box.innerHTML = `<p>Por favor, corrige lo siguiente:</p><ul>${items}</ul>`;
}

function passwordStrength(value, minLen){
  let s = 0;
  if (value.length >= minLen) s++;
  if (/[A-Z]/.test(value)) s++;
  if (/\\d/.test(value)) s++;
  if (/^[A-Za-z0-9]/.test(value)) s++;
  return s; // 0..4
}

document.addEventListener('DOMContentLoaded', () => {
  const form      = document.getElementById('f-registro');
  const fields     = form.querySelectorAll('input, select, textarea');

  const email      = document.getElementById('email');
  const emailStatus= document.getElementById('email-status');
  let emailTimer = 0, emailQuery = 0;

  const pass       = document.getElementById('pass');
  const pass2      = document.getElementById('pass2');
  const passMeter  = document.getElementById('pass-strength');
  const passText   = document.getElementById('pass-strength-text');
  const passHint   = document.getElementById('hint-pass');

  // ===== Config global de contraseña =====
  const DESIRED_MIN = 8; // cambia a 12/16 si quieres
  // Quita cualquier minlength que venga del HTML o de otro script
  if (pass.hasAttribute('minlength')) pass.removeAttribute('minlength');
  // Refleja el mínimo en el hint
  if (passHint) passHint.textContent =
    `Mínimo ${DESIRED_MIN} caracteres. Combina mayúsculas, números y símbolos.`;

  // ===== Validación nativa para todo MENOS 'pass' (que controlamos nosotros) =====
  fields.forEach(el => {
    if (el.id === 'pass') return;

```

```

    el.addEventListener('input', () => showError(el, nativeMessage(el)));
    el.addEventListener('blur', () => showError(el, nativeMessage(el)));
  });

  // ===== Regla de longitud propia para 'pass' =====
  function checkPasswordMin(){
    const rest = DESIRED_MIN - pass.value.length;
    if (rest > 0){
      const msg = `Mínimo ${DESIRED_MIN} caracteres. Añade ${rest}
${plural(rest, 'carácter', 'caracteres')}`;
      pass.setCustomValidity(msg);
      showError(pass, msg);
    } else {
      pass.setCustomValidity('');
      showError(pass, '');
    }
  }

  // Fuerza y mensajes accesibles
  pass.addEventListener('input', () => {
    checkPasswordMin(); // primero nuestra regla
    const score = passwordStrength(pass.value, DESIRED_MIN);
    passMeter.value = score;
    const textos = ['Muy débil', 'Débil', 'Aceptable', 'Buena', 'Fuerte'];
    passText.textContent = pass.value ? `Fuerza: ${textos[score]}` : '';
  });
  pass.addEventListener('blur', checkPasswordMin);

  // Confirmación de contraseña
  function checkPasswords(){
    pass2.setCustomValidity(''); showError(pass2, '');
    if (pass.value && pass2.value && pass.value !== pass2.value){
      const msg = 'La confirmación no coincide con la contraseña.';
      pass2.setCustomValidity(msg); showError(pass2, msg);
    }
  }
  pass.addEventListener('input', checkPasswords);
  pass2.addEventListener('input', checkPasswords);

  // Validación asíncrona simulada para correo (debounce 300 ms)
  email.addEventListener('input', () => {
    clearTimeout(emailTimer);
    emailStatus.textContent = '';
    if (!email.checkValidity()) return;

    emailTimer = setTimeout(() => {
      const qid = ++emailQuery;
      emailStatus.textContent = 'Comprobando disponibilidad...';
    }, 300);
  });

```



```

// Simulación: "existe" si incluye "test"
const exists = /test/i.test(email.value);

setTimeout(() => {
  if (qid !== emailQuery) return; // respuesta vieja
  if (exists){
    const msg = 'Este correo ya está registrado. ¿Quieres iniciar sesión?';
    email.setCustomValidity(msg);
    showError(email, msg);
    emailStatus.textContent = 'Ya registrado.';
  } else {
    email.setCustomValidity('');
    showError(email, '');
    emailStatus.textContent = 'Disponible.';
  }
}, 300);
}, 300);
});

// Envío: muestra resumen y enfoca el primer error
form.addEventListener('submit', (e) => {
  const invalid = [];

  // Revisa todos menos 'pass' con nativo
  for (const el of fields){
    if (el.id === 'pass') continue;
    const msg = nativeMessage(el);
    showError(el, msg);
    if (msg) invalid.push({el, msg});
  }

  // Reglas propias
  checkPasswordMin();
  checkPasswords();
  if (!pass.validity.valid) invalid.push({el: pass, msg: pass.validationMessage});
  if (!pass2.validity.valid) invalid.push({el: pass2, msg: pass2.validationMessage});

  if (invalid.length){
    e.preventDefault();
    renderSummary(form, invalid);
    invalid[0].el.focus();
  } else {
    renderSummary(form, []);
    // aquí enviaría normalmente
  }
});
});

```

5) Pruebas rápidas

- **Teclado:** completa el flujo sin mouse; el foco debe ser claro y el orden lógico.
- **Errores:** fuerza 3–4 errores; deben aparecer junto a los campos y además en el **resumen** superior, con anclas que te llevan al campo.
- **Nativo:** email mal formado → borde rojo por estados :invalid.
- **JS:** contraseñas distintas → error en confirmación + resumen.
- **Asíncrono (simulado):** escribe un correo con “test” → “ya registrado” + bloqueo del envío.
- **Accesibilidad:** el resumen tiene role="alert", el estado del email usa role="status", y el foco salta al **primer** error al enviar.

Ejercicios para resolver en clase (15–20 min)

Ejercicio 1 — Campo “Identificación” con patrón local

Añade un campo “Identificación nacional (10 dígitos)” antes del teléfono:

- `type="text", pattern="^\d{10}$", inputmode="numeric".`
- Hint con ejemplo y error vinculado; integra el campo a la validación (verás el mensaje nativo + el personalizado si quieres).

Criterio de finalización: el error aparece junto al campo y en el resumen; el envío se bloquea si no cumple.

Ejercicio 2 — Select de provincia + código postal

Agrega select de provincia (3–5 opciones) y un input postal-code (`minlength="4" maxlength="6", inputmode="numeric"`).

- Usa tokens de autocomplete (`address-level1, postal-code`).
- Añade validación nativa y mensajes claros.

Criterio de finalización: con valores inválidos, se muestran errores vinculados y el resumen; con datos válidos, el formulario envía.